

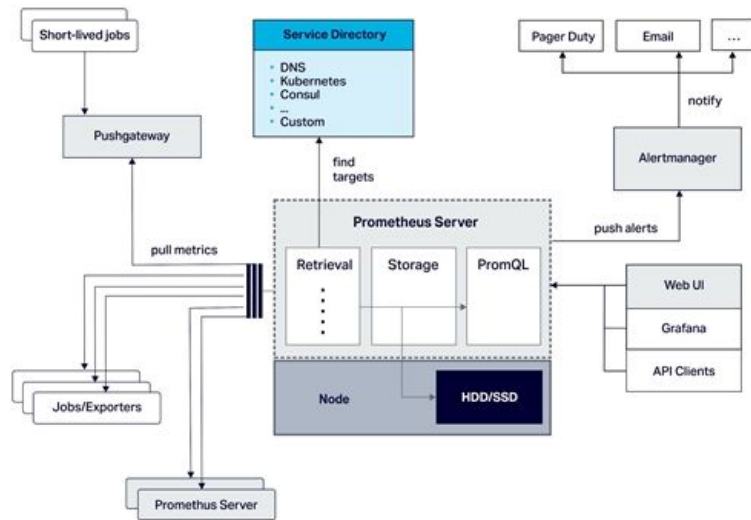


Моніторинг сервісу за допомогою Prometheus

Розуміння системи моніторингу та оповіщення Prometheus

Спочатку розроблений SoundCloud, потім створений з відкритим кодом і прийнятий як другий проект у Cloud Native Computing Foundation (CNCF) у 2016 році, Prometheus стає все більш популярним інструментом, що забезпечує моніторинг і сповіщення для програм і серверів.

Архітектура Prometheus виглядає так:



Компоненти

Основні компоненти Prometheus включають сервер Prometheus (який обслуговує виявлення служб, отримання метрик із відстежуваних програм, зберігання цих метрик і аналіз даних часових рядів за допомогою мови запитів PromQL), модель даних метрики, простий вбудований графічний інтерфейс користувача та вбудовану підтримку Grafana. Додаткові компоненти включають диспетчер сповіщень (у якому сповіщення можна визначити на мові запитів) і шлюз push, корисний для моніторингу короткострокових програм.

Зазвичай інструменти для моніторингу програм збирають показники за допомогою одного з цих трьох методів:

- **Інструменти:** додавання спеціального коду до вихідного коду контрольованої програми.
- **Агенти:** додавання спеціального коду загального призначення до середовища програми, призначеного для автоматичного збору стандартних показників.
- **Шпигунство:** використання перехоплювачів або мережевих кранів для моніторингу викликів або потоку даних між системами.

Компоненти

Prometheus підтримує використання комбінації інструментів і агентів (які він називає «експортерами»). Інструментам потрібен доступ до вихідного коду, і він дає змогу отримувати спеціальні показники. Він також не залежить від мови програмування та має офіційно підтримувані клієнтські бібліотеки, доступні для Go, Java/Scala, Python і Ruby. Також доступно багато неофіційних бібліотек (LISP тощо), або ви можете написати власну.

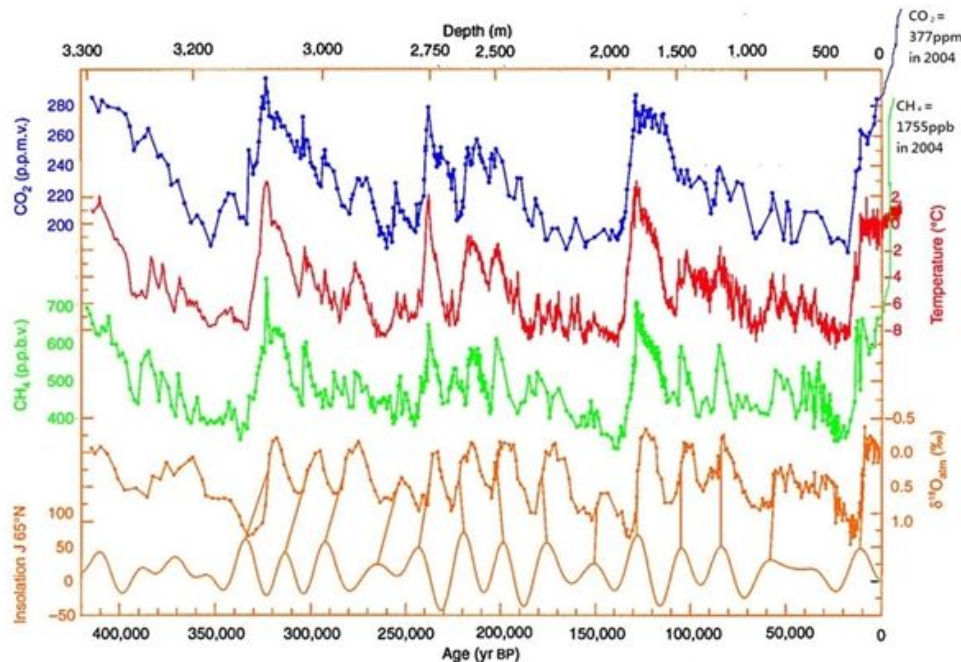
Багато сторонніх експортерів доступні для ввімкнення автоматичного інструментарію певного програмного забезпечення, включаючи популярні бази даних, апаратне забезпечення, системи обміну повідомленнями, сховище, HTTP, хмарні API, журналювання, системи моніторингу тощо. Доступний експортер JMX може експортувати показники для додатків на основі JVM, таких як Kafka і Cassandra. У той же час, деяке програмне забезпечення відображатиме показники у форматі Prometheus, що зробить непотрібними експортери.

Для моніторингу апаратного забезпечення хоста та показників ядра доступний експортер вузла. Клієнт Prometheus Java містить збирачі для збирання сміття, пули пам'яті, JMX, завантаження класів і підрахунки потоків, які можна додавати по одному або реєструвати всі одночасно за допомогою `DefaultExports.initialize()`;

Що Prometheus робить добре (а що не робить зовсім)

Prometheus чудово підходить для моніторингу показників – і все. Це не ефективний інструмент керування продуктивністю додатків (APM), оскільки він зосереджений виключно на показниках на стороні сервера. Він не пропонує розподілене відстеження викликів, виявлення та візуалізацію топології служби, аналітику продуктивності або моніторинг досвіду кінцевих користувачів (хоча є розширення github, яке може передавати показники браузера користувача в Prometheus). Потік інформації з Prometheus є одностороннім, тому його не можна використовувати для активного контролю. Нарешті, Prometheus за замовчуванням працює як один сервер, але його можна масштабувати за допомогою об'єднання серверів.

Моделювання даних Prometheus



Мал. 420 000 років даних про
часові ряди ядра
антарктичного льоду

Моделювання даних Prometheus

Prometheus зберігає метрики як дані часових рядів, так що метрики включають потоки значень (64-розрядне число з плаваючою точкою) з мітками часу з точністю до мілісекунди. Кожна метрика має назву (рядок) і використовує угоду про найменування, яка включає назву того, що відстежується, логічний тип і одиниці вимірювання. Кожен показник також має набір пар ключ:значення (розмічені параметри). Наприклад, метрика з назвою `http_requests_total` може включати мітки для «методу» («GET», «PUT») і для «обробника» (наприклад: «/login», «/search»). Prometheus також автоматично додає деякі мітки до показників, зокрема:

- завдання: налаштована назва завдання, до якого належить ціль.
- Примірник: `<host>:<port>` частина URL-адреси, отримана з цільового.

Оскільки одиниці не є явними, перетворення з однієї одиниці в іншу можна виконати лише вручну на мові запитів (і слід виконувати дуже обережно).

Типи метрик

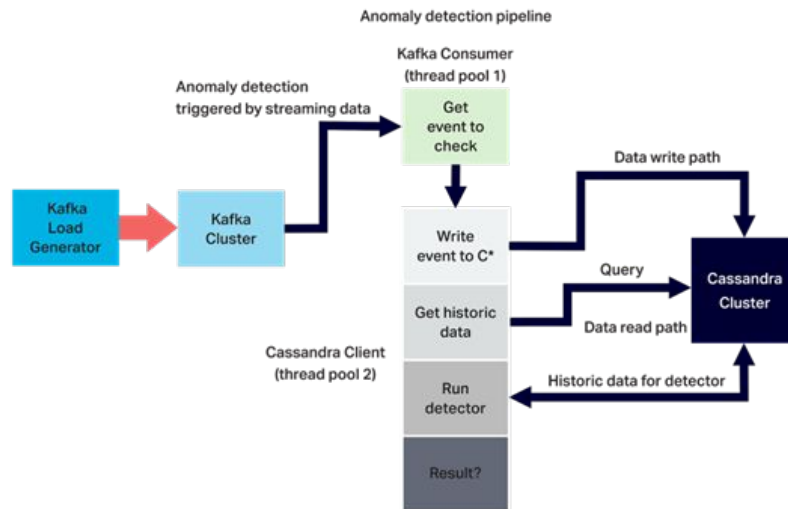
Prometheus пропонує ці чотири різні типи показників:

1. Лічильник: лічильник корисний для значень, які можуть лише збільшуватися (значення можна скинути до нуля під час перезавантаження).
2. Датчик: вимірювальний показник можна використовувати для підрахунку, який зростає та зменшується, а також корисний для виміряних значень, які можуть зростати та падати.
3. Гістограма: гістограма містить вибірку спостережень, як-от тривалість запиту або розмір відповіді. Він підраховує їх у сегментах, які ви можете налаштувати, а також пропонує суму всіх спостережених значень.
4. Підсумок: Подібно до гістограми, підсумкові вибірки спостережень пропонують загальну кількість спостережень і суму спостережуваних значень, а також обчислюють конфігуровані квантилі протягом ковзного часового вікна.

Запуск Prometheus для моніторингу Anomalia Machina

Щоб зрозуміти, що ми нещодавно хотіли контролювати за допомогою експерименту Anomalia Machina, давайте подивимося на його функціональну архітектурну схему:

Functional Architecture: Prototype



Запуск Prometheus для моніторингу Anomalia Machina

Використовуючи Prometheus, ми відслідковували «загальні» показники програми, включаючи пропускну здатність (TPS) і час відгуку генератора навантаження Kafka (виробник Kafka), споживача Kafka та клієнта Cassandra (який виявляє аномалії). Крім того, ми хотіли відстежувати деякі специфічні для програми показники, зокрема кількість рядків, що повертаються для кожного читання Cassandra, і кількість виявлених аномалій. Нам також потрібно було відстежувати апаратні показники, такі як ЦП, для кожного екземпляра AWS EC2, на якому працює програма, а також централізувати моніторинг, додавши туди показники Kafka та Cassandra.

Щоб досягти цього, ми почали зі створення простого тестового конвеєра з трьома методами (виробник, споживач і детектор). Потім ми використали метрику лічильника під назвою «prometheusTest_requests_total», щоб виміряти, скільки разів кожен етап конвеєра виконується успішно, і мітку під назвою «stage», щоб розрізнити різні етапи підрахунку (використовуючи «total» для загальної кількості конвеєрів). Потім ми використали другий лічильник під назвою «prometheusTest_anomalies_total» для підрахунку виявлених аномалій. І ми використали датчик під назвою «prometheusTest_duration_seconds», щоб записати тривалість кожного етапу в секундах (використовуючи мітку «стадія», щоб розрізнити етапи, і мітку «загальна» для загальної тривалості конвеєра).

Запуск Prometheus для моніторингу Anomalia Machina

Методи мають інструменти, які збільшують показники лічильника кожного разу, коли етап успішно виконується або виявляється аномалія (за допомогою методу `inc()`), і встановлюють значення часу вимірювального показника для кожного етапу (за допомогою методу `setToTime()`).

Ось приклад коду:

```
import java.io.IOException;
import io.prometheus.client.Counter;
import io.prometheus.client.Gauge;
import io.prometheus.client.exporter.HTTPServer;
import io.prometheus.client.hotspot.DefaultExports;

// https://github.com/prometheus/client_java
// Demo of how we plan to use Prometheus Java client to instrument Anomalia Machina
// Note that the Anomalia Machina application will have Kafka Producer and Kafka Consumer
// So metrics from each will have different host/port combinations.
public class PrometheusBlog {
    static String appName = "prometheusTest";
    // counters can only increase in value (until process restart)
    // Execution count. Use a single Counter for all stages of the pipeline, stages are
    static final Counter pipelineCounter = Counter.build()
        .name(appName + "_requests_total").help("Count of executions of pipeline stages")
        .labelNames("stage")
        .register();
    // in theory could also use pipelineCounter to count anomalies found using another counter
    // but less potential for confusion having another counter. Doesn't need a label
    static final Counter anomalyCounter = Counter.build()
        .name(appName + "_anomalies_total").help("Count of anomalies detected")
        .register();
    // A Gauge can go up and down, and is used to measure current value of some variable
    // pipelineGauge will measure duration in seconds of each stage using labels.
    static final Gauge pipelineGauge = Gauge.build()
        .name(appName + "_duration_seconds").help("Gauge of stage durations in seconds")
        .labelNames("stage")
        .register();

    public static void main(String[] args) {
        // Allow default JVM metrics to be exported
        DefaultExports.initialize();

        // Metrics are pulled by Prometheus, create an HTTP server as the endpoint
        // Note if there are multiple processes running on the same server need to change port
        // And add all IPs and port numbers to the Prometheus configuration file.
        HTTPServer server = null;
        try {
            server = new HTTPServer(1234);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Запуск Prometheus для моніторингу Anomalia Machina

Ось приклад коду
(Продовження):

```
// now run 1000 executions of the complete pipeline with random time delays and i
int max = 1000;
for (int i=0; i < max; i++)
{
    // total time for complete pipeline, and increment anomalyCounter
    pipelineGauge.labels("total").setToTime(() -> {
        producer();
        consumer();
        if (detector())
            anomalyCounter.inc();
    });
    // total pipeline count
    pipelineCounter.labels("total").inc();
    System.out.println("i=" + i);

    // increase the rate of execution
    try {
        Thread.sleep(max-i);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
server.stop();

// the 3 stages of the pipeline, for each we increase the stage counter and set t
public static void producer() {
    class Local {};
    String name = Local.class.getEnclosingMethod().getName();
    pipelineGauge.labels(name).setToTime(() -> {
        try {
            Thread.sleep(1 + (long)(Math.random()*20));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
    pipelineCounter.labels(name).inc();
}

public static void consumer() {
    class Local {};
    String name = Local.class.getEnclosingMethod().getName();
    pipelineGauge.labels(name).setToTime(() -> {
        try {
            Thread.sleep(1 + (long)(Math.random()*10));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
}
```



```
pipelineCounter.labels(name).inc();
}

// detector returns true if anomaly detected else false
public static boolean detector() {
    class Local {};
    String name = Local.class.getEnclosingMethod().getName();
    pipelineGauge.labels(name).setToTime(() -> {
        try {
            Thread.sleep(1 + (long)(Math.random()*200));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
    pipelineCounter.labels(name).inc();
    return (Math.random() > 0.95);
}
}
```

Запуск Prometheus для моніторингу Anomalia Machina

З підготовленим зразком коду важливо зрозуміти, як запускати Prometheus і як Prometheus отримує метричні значення з коду. На відміну від корпоративних рішень АРМ, у які надсилаються метрики, Prometheus отримує метрики шляхом опитування («збирання») інструментального коду. Для цього просто потрібен HTTP-сервер для роботи в коді програми: наведений вище код створює HTTP-сервер на порту 1234, який дозволяє Prometheus очищати метрики.

Далі нам потрібно було розглянути залежності Maven:

```
<!-- The client -->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient</artifactId>
  <version>LATEST</version>
</dependency>
<!-- Hotspot JVM metrics-->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient_hotspot</artifactId>
  <version>LATEST</version>
</dependency>
<!-- Exposition HTTPServer-->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient_httpserver</artifactId>
  <version>LATEST</version>
</dependency>
<!-- Pushgateway exposition-->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient_pushgateway</artifactId>
  <version>LATEST</version>
</dependency>
```

Запуск Prometheus для моніторингу Anomalia Machina

І, нарешті, нам потрібно було сказати Prometheus, звідки він береться. Для простого розгортання чи тестування доцільно додати цю інформацію до файлу конфігурації (за замовчуванням це prometheus.yml):

```
global:
  scrape_interval:     15s # By default, scrape targets every 15 seconds.

# scrape_configs has jobs and targets to scrape for each.
scrape_configs:
  # job 1 is for testing prometheus instrumentation from multiple application processes.
  # The job name is added as a label job=<job_name> to any timeseries scraped from this config.
  - job_name: 'testprometheus'

    # Override the global default and scrape targets from this job every 5 seconds.
    scrape_interval: 5s

    # this is where to put multiple targets, e.g. for Kafka load generators and detectors
    static_configs:
      - targets: ['localhost:1234', 'localhost:1235']

  # job 2 provides operating system metrics (e.g. CPU, memory etc).
  - job_name: 'node'

    # Override the global default and scrape targets from this job every 5 seconds.
    scrape_interval: 5s

    static_configs:
      - targets: ['localhost:9100']
```

Запуск Prometheus для моніторингу Anomalia Machina

Файл конфігурації також містить завдання під назвою «вузол» із використанням порту 9100. Це завдання надає метрику вузла — для його використання потрібно завантажити програму експорту вузлів Prometheus і запустити її на тому самому сервері, що й програма. Опитування метрик має плюси та мінуси: надто часте опитування може перевантажувати програми, а недостатньо часте опитування може спричинити тривожні затримки між моментом виникнення подій та моментом їх виявлення.

У той же час ця система є надійною та досить слабко пов'язаною, оскільки програми можуть працювати без Prometheus, і Prometheus просто продовжуватиме опитувати недоступні програми, доки вони не стануть доступними. Також можна опитувати одну програму з кількома серверами Prometheus. Але якщо з будь-якої причини неможливо опитувати показники програми або програма дуже швидкоплинна, Prometheus пропонує шлюз push, який можна використовувати як альтернативу.

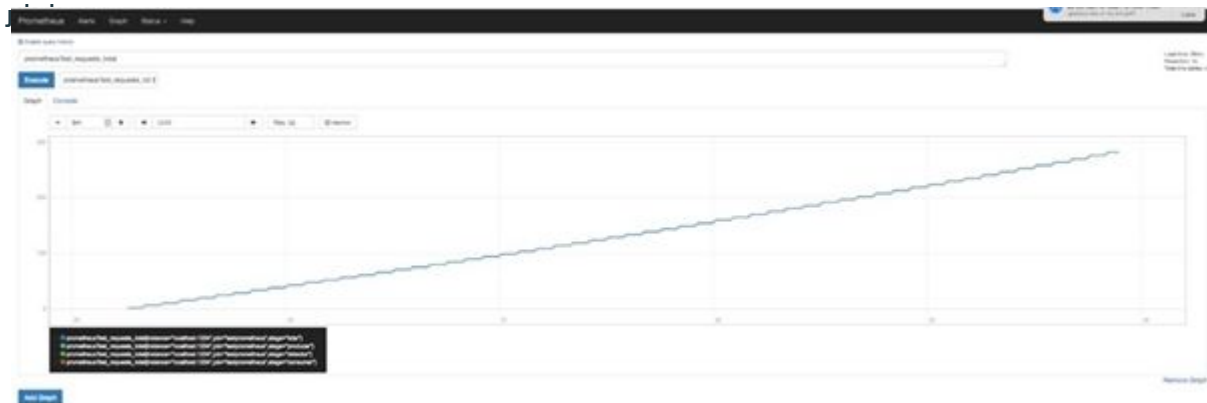
Початкові результати за допомогою Prometheus

Prometheus не включає інформаційні панелі за замовчуванням, тому спочатку ми використовували вирази в нашому експерименті. В інтерфейсі Prometheus ви знайдете меню прокручування вниз, у якому можна вибрати назви показників (ви також можете зробити це через браузер за адресою <http://localhost:9090/metrics>). Потім ви можете ввести ці показники у поле виразу та виконати їх. Не засмучуйтесь, якщо ви зіткнетеся з повідомленням про помилку та потребуєте вирішення певної проблеми на цьому етапі (це типова ситуація).

Коли робочий вираз створено, ви можете переглянути результати в таблиці або на графіку, якщо доступний для цього типу результату. За замовчуванням вирази повертаються лише на п'ять хвилин, щоб знайти дані, і якщо дані недоступні, ви отримаєте повідомлення про помилку. Щоб спробувати Prometheus, ви можете скористатися тим фактом, що Prometheus стежить за собою, щоб дослідити рішення, не потребуючи наявності інструментальної програми.

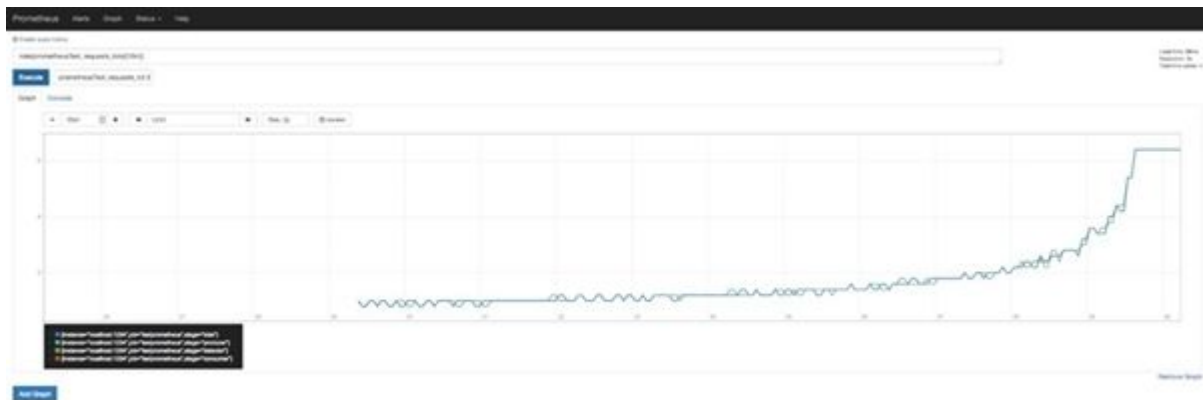
Графічні дані

Оскільки показник лічильника просто збільшує значення, побудова графіка лічильника просто створить



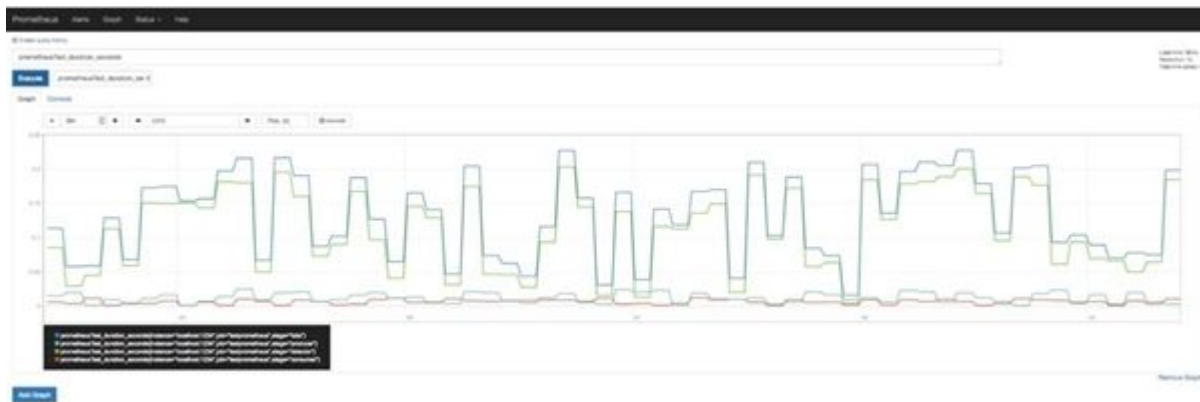
Графічні дані

Щоб отримати графік ставок на основі показника лічильника, скористайтесь функцією `irate` або `rate`.



Графічні дані

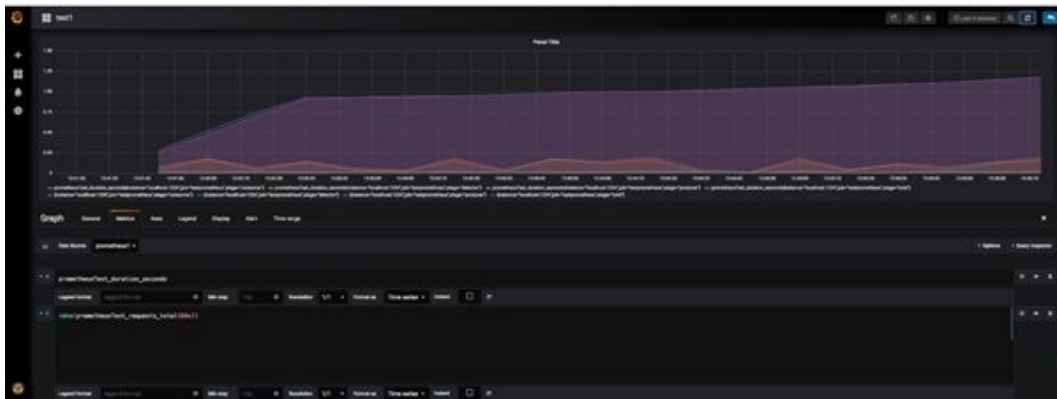
У наведеному нижче прикладі графік відображає тривалість етапів конвеєра та не потребує функції швидкості, оскільки це вимірювальний показник, а не лічильник:



Графічні дані

Хоча вбудовані можливості Prometheus для побудови графіків обмежені – наприклад, ви не можете побудувати більше одного показника на одному графіку – додавання Grafana пропонує значно розширені можливості. Grafana пропонує вбудовану підтримку Prometheus і є обов'язковим для серйозного використання графіків. Сайти Prometheus і Grafana пропонують корисну документацію щодо спільного використання цих інструментів.

Щоб використовувати Grafana, встановіть його та перейдіть у веб-переглядачі за адресою <http://localhost:3000/>. Створіть джерело даних Prometheus, а потім додайте графік Prometheus, використовуючи вираз, як зазвичай. Наприклад, наведений нижче графік може відображати показники тривалості та швидкості:



Графічні дані

Одна порада: якщо на графіку нічого не видно, ймовірно, проблема в тому, що ви не бачите правильний діапазон часу. Швидким рішенням є використання «Швидкого діапазону» — налаштування «Останні 5 хвилин» має бути достатнім. Ви також можете використовувати правила для попереднього обчислення ставок, щоб потенційно прискорити агрегацію.

Також можна побудувати графічні показники вузла, наприклад використання ЦП. Наприклад, ви можете обчислити використання ЦП у відсотках за допомогою цього виразу:

```
100 - (avg by (instance) (irate(node_cpu_seconds_total{job="node",mode="idle"}[5m])) * 100)
```

Підсумок

Нарешті, ті, хто використовує Cassandra з цими інструментами, також можуть знайти користь у використанні Instacluster's Cassandra Exporter for Prometheus, який ідеально підходить для інтеграції метрик Cassandra з самокерованого кластера у ваш моніторинг програми за допомогою Prometheus. Інструмент добре задокументований і є корисною відправною точкою для глибшого розуміння Prometheus.

Отримавши знання про те, як використовувати Prometheus для моніторингу зразка програми, ми можемо контролювати фактичний код програми Machine Anomalia та впевнено запускати його у виробництво.

Питання та відповіді